

LABORATORY OBSERVATION/RECORD

Course:	Full Stack Web Development
Course Code:	23CM4121
Experiment No.:	3
Student Name:	P.Govindamma
Roll No.:	A24126552104
Date:	29-08-2026

1. TITLE

Develop a Dynamic To-Do List Application Using JavaScript, DOM Manipulation and Event Listeners

2. AIM

To develop a dynamic To-Do List application using JavaScript, DOM manipulation, and event listeners that allows users to add, complete, and delete tasks dynamically from a webpage.

3. OBJECTIVE

The objectives of this experiment are:

- To understand JavaScript DOM manipulation.
- To access HTML elements using JavaScript.
- To handle user actions using event listeners.
- To dynamically create new HTML elements.
- To add new tasks to a list dynamically.
- To implement a Complete button for each task.
- To visually indicate completed tasks.
- To implement a Delete button for each task.
- To remove individual tasks dynamically from the webpage.
- To display an appropriate message when no tasks are available.
- To handle the Enter key for adding tasks.
- To apply CSS styling to the To-Do List application.

4. THEORY

JavaScript is a programming language used to add dynamic behavior and interactivity to web pages. It can respond to user actions and modify webpage content without requiring the page to be reloaded. The DOM (Document Object Model) represents an HTML document as a tree of objects. JavaScript can access, create, modify, and remove HTML elements using DOM methods. In this application, JavaScript is used to dynamically create task items and add them to an unordered list.

DOM Manipulation

DOM manipulation allows JavaScript to dynamically modify webpage content. In this experiment, methods such as `getElementById()`, `createElement()`, `appendChild()`, and `remove()` are used.

Event Listeners

The `addEventListener()` method is used to perform an action when a particular event occurs. In this application, click events are used for the Add Task, Complete, and Delete buttons.

Dynamic Task Creation

When the user enters a task and clicks the Add Task button, JavaScript creates a new `<li>` element dynamically and adds it to the task list.

Complete Functionality

The Complete button uses `classList.toggle()` to add or remove the completed CSS class. The class applies a line-through effect to indicate that the task has been completed. The button label also updates between "Complete" and "Completed" depending on the task's state.

Delete Functionality

The Delete button uses the `remove()` method to remove the corresponding task from the webpage.

Empty List Message

The program checks the number of child elements in the task list using `taskList.children.length`. If there are no tasks, the message "No tasks available." is displayed.

The basic flow of the application is:

User Input → Add Task → DOM Element Creation → Task List → Complete/Delete → DOM Update

5. ALGORITHM / PROCEDURE

- 1 Create an HTML5 document.
- 2 Add a heading titled My To-Do List.
- 3 Create a text box for entering tasks.
- 4 Create an Add Task button.
- 5 Create a message to indicate that no tasks are available.
- 6 Create an unordered list to display tasks.
- 7 Create a JavaScript file.
- 8 Access the required HTML elements using `getElementById()`.
- 9 Create a function to update the empty-list message.
- 10 Add a click event listener to the Add Task button.
- 11 Read the task entered by the user.
- 12 Remove unnecessary spaces using `trim()`.
- 13 Check whether the task input is empty.
- 14 Display an alert if the input is empty.
- 15 Create a new `<li>` element dynamically and assign it the "task" class.
- 16 Create a `<span>` element for the task text with the "task-name" class.
- 17 Create a Complete button dynamically.
- 18 Add a click event listener to the Complete button.

- 19 Toggle the completed CSS class and update the button label when the Complete button is clicked.
- 20 Create a Delete button dynamically.
- 21 Add a click event listener to the Delete button.
- 22 Remove the corresponding task when Delete is clicked.
- 23 Add the task text and buttons to the task item.
- 24 Add the task item to the task list.
- 25 Clear the input field after adding the task.
- 26 Update the empty-list message.
- 27 Add an event listener for the Enter key (keydown).
- 28 Allow the Enter key to trigger the Add Task button.
- 29 Call the empty-message function when the page initially loads.
- 30 Apply CSS styling to the webpage.
- 31 Test adding, completing, and deleting tasks.
- 32 Verify the final output.

6. PROGRAM

toDoList.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My To-Do List</title>
  <link rel="stylesheet" href="toDoList.css">
</head>
<body>
  <div class="container">
    <h1>My To-Do List</h1>
    <div class="input-section">
      <input type="text" id="taskInput" placeholder="Enter a task...">
      <button id="addBtn">Add Task</button>
    </div>
    <p id="emptyMessage">No tasks available.</p>
    <ul id="taskList"></ul>
  </div>
  <script src="toDoList.js"></script>
</body>
</html>
```

toDoList.css

```
* {
  box-sizing: border-box;
}
body {
  font-family: Arial, sans-serif;
  background-color: #f2f2f2;
  margin: 0;
  padding: 40px;
}
.container {
  width: 500px;
  margin: auto;
  background-color: white;
  padding: 25px;
  border-radius: 10px;
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.15);
}
h1 {
```

```

        text-align: center;
        color: #333;
    }
    .input-section {
        display: flex;
        gap: 10px;
    }
    #taskInput {
        flex: 1;
        padding: 10px;
        font-size: 16px;
        border: 1px solid #aaa;
        border-radius: 5px;
    }
    #addBtn {
        padding: 10px 15px;
        background-color: #007bff;
        color: white;
        border: none;
        border-radius: 5px;
        cursor: pointer;
    }
    #addBtn:hover {
        background-color: #0056b3;
    }
    #emptyMessage {
        text-align: center;
        color: #777;
        margin-top: 25px;
    }
    #taskList {
        list-style: none;
        padding: 0;
        margin-top: 20px;
    }
    .task {
        display: flex;
        align-items: center;
        gap: 8px;
        padding: 12px;
        margin-bottom: 10px;
        background-color: #eeeeee;
        border-radius: 5px;
    }
    .task-name {
        flex: 1;
        font-size: 16px;
    }
    .completed {
        text-decoration: line-through;
        color: gray;
    }
    .complete-btn {
        background-color: #28a745;
        color: white;
        border: none;
        padding: 7px 10px;
        border-radius: 4px;
        cursor: pointer;
    }
    .delete-btn {
        background-color: #dc3545;
        color: white;
        border: none;
        padding: 7px 10px;
        border-radius: 4px;
        cursor: pointer;
    }
    .complete-btn:hover {
        background-color: #218838;
    }
    .delete-btn:hover {
        background-color: #c82333;
    }
}

```

todoList.js

```

const taskInput = document.getElementById("taskInput");

```

```

const addBtn = document.getElementById("addBtn");
const taskList = document.getElementById("taskList");
const emptyMessage = document.getElementById("emptyMessage");

// Function to update empty message
function updateMessage() {
    if (taskList.children.length === 0) {
        emptyMessage.style.display = "block";
    } else {
        emptyMessage.style.display = "none";
    }
}

// Add task
addBtn.addEventListener("click", function () {
    const taskValue = taskInput.value.trim();

    if (taskValue === "") {
        alert("Please enter a task.");
        return;
    }

    // Create list item
    const taskItem = document.createElement("li");
    taskItem.classList.add("task");

    // Create task text
    const taskName = document.createElement("span");
    taskName.textContent = taskValue;
    taskName.classList.add("task-name");

    // Create Complete button
    const completeBtn = document.createElement("button");
    completeBtn.textContent = "Complete";
    completeBtn.classList.add("complete-btn");

    // Complete button event
    completeBtn.addEventListener("click", function () {
        taskName.classList.toggle("completed");
        if (taskName.classList.contains("completed")) {
            completeBtn.textContent = "Completed";
        } else {
            completeBtn.textContent = "Complete";
        }
    });

    // Create Delete button
    const deleteBtn = document.createElement("button");
    deleteBtn.textContent = "Delete";
    deleteBtn.classList.add("delete-btn");

    // Delete button event
    deleteBtn.addEventListener("click", function () {
        taskItem.remove();
        updateMessage();
    });

    // Add elements to task item
    taskItem.appendChild(taskName);
    taskItem.appendChild(completeBtn);
    taskItem.appendChild(deleteBtn);

    // Add task to the list
    taskList.appendChild(taskItem);

    // Clear input
    taskInput.value = "";

    // Update empty message
    updateMessage();
});

// Add task when Enter key is pressed
taskInput.addEventListener("keydown", function (event) {
    if (event.key === "Enter") {
        addBtn.click();
    }
});
// Show empty message initially

```

```
updateMessage();
```

7. CODE EXPLANATION

Code	Explanation
<code>getElementById()</code>	Selects an HTML element using its ID.
<code>addEventListener()</code>	Attaches an event handler to an HTML element.
<code>taskInput.value</code>	Retrieves the task entered by the user.
<code>trim()</code>	Removes unnecessary spaces from the beginning and end of the input.
<code>alert()</code>	Displays a message to the user.
<code>createElement()</code>	Dynamically creates a new HTML element.
<code>classList.add()</code>	Assigns one or more CSS classes to an element.
<code>textContent</code>	Sets the text content of an element.
<code>appendChild()</code>	Adds a dynamically created element to another element.
<code>classList.toggle()</code>	Adds or removes the specified CSS class.
<code>classList.contains()</code>	Checks whether an element currently has a given CSS class.
<code>remove()</code>	Removes an HTML element from the webpage.
<code>children.length</code>	Determines the number of child elements in the task list.
<code>style.display</code>	Controls whether the empty-list message is displayed.
<code>keydown</code>	Detects keyboard input events.
<code>event.key</code>	Identifies the key pressed by the user.
<code>.completed</code>	CSS class used to visually indicate a completed task.
<code>:hover</code>	Applies CSS styling when the pointer moves over an element.

8. TEST CASES AND EXECUTION DATA

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Open the webpage	To-Do List interface should be displayed	Interface displayed correctly	Pass
TC-02	Page loaded with no tasks	"No tasks available." should be displayed	Message displayed correctly	Pass
TC-03	Enter "Complete Assignment" and click Add Task	Task should be added to the list	Task added successfully	Pass
TC-04	Enter "Prepare for Quiz" and click Add Task	New task should be added	Task added successfully	Pass

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-05	Enter "Practice JavaScript" and click Add Task	New task should be added	Task added successfully	Pass
TC-06	Click Complete for a task	Task should be visually marked as completed	Line-through effect displayed and button label changed to "Completed"	Pass
TC-07	Click Delete for a task	Selected task should be removed	Task removed successfully	Pass
TC-08	Delete all tasks	"No tasks available." should appear	Message displayed correctly	Pass
TC-09	Click Add Task with empty input	Alert should request the user to enter a task	Alert displayed correctly	Pass
TC-10	Enter a task and press Enter	Task should be added to the list	Task added successfully	Pass

9. OUTPUT

Output : To-Do List After Adding Tasks

The following tasks are entered during execution:

- Complete Assignment
- Prepare for Quiz
- Practice JavaScript

The resulting webpage displays:

Complete Assignment — **Complete** — **Delete**

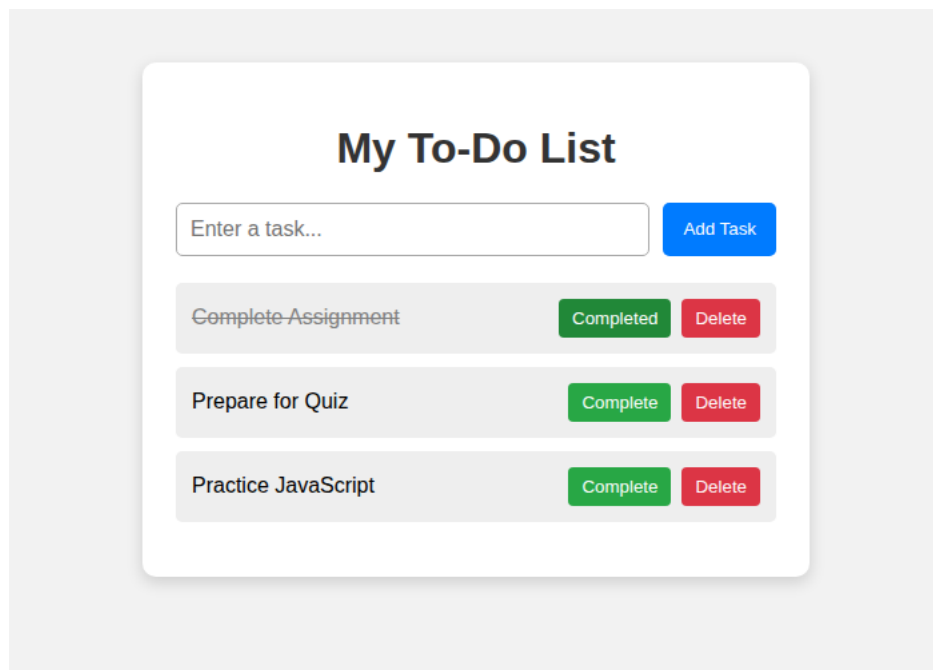
Prepare for Quiz — **Complete** — **Delete**

Practice JavaScript — **Complete** — **Delete**

When the **Complete** button is clicked, the corresponding task text is displayed with a **line-through effect** and the button label changes to "**Completed**".

When the **Delete** button is clicked, the selected task is removed from the list.

When all tasks are deleted, the "**No tasks available.**" message reappears below the input section.



Screenshot: To-Do List after adding three tasks and marking "Complete Assignment" as completed.

10. OBSERVATION / INTERVIEW QUESTIONS

Q1. What is DOM manipulation?

Answer: DOM manipulation is the process of accessing and modifying HTML elements dynamically using JavaScript. It allows elements to be created, changed, or removed without manually modifying the HTML document.

Q2. What is the purpose of `addEventListener()`?

Answer: The `addEventListener()` method is used to attach an event handler to an HTML element. In this application, it is used to respond to button clicks and keyboard events.

Q3. What is the purpose of `createElement()`?

Answer: The `createElement()` method dynamically creates a new HTML element. In this application, it is used to create task items, task text, Complete buttons, and Delete buttons.

Q4. How does the Complete button work?

Answer: When the Complete button is clicked, the program uses `classList.toggle("completed")` on the task text. This adds or removes the completed CSS class, which applies a line-through effect, and the button's own label is updated between "Complete" and "Completed" using `classList.contains()`.

Q5. How does the Delete button remove a task?

Answer: The Delete button calls the `remove()` method on the corresponding task item (the `<li>` element). This removes the selected task from the webpage.

Q6. How is the "No tasks available" message displayed?

Answer: The `updateMessage()` function checks `taskList.children.length`. If the length is zero, the empty message is displayed. Otherwise, it is hidden.

Q7. What happens when the user tries to add an empty task?

Answer: The program checks the entered value using `trim()`. If the value is empty, an alert message "Please enter a task." is displayed and the task is not added.

11. RESULT

Thus, a dynamic To-Do List application was successfully developed using JavaScript, DOM manipulation, and event listeners. Tasks were dynamically added to the list, individual tasks could be marked as completed with both a visual line-through effect and an updated button label, and tasks could be deleted from the webpage. The application also displayed an appropriate message when the task list was empty and was successfully verified using multiple test cases.